CS543 · WEB SERVICES · ASSIGNMENT 7 OF 12

Build one CampusEats service in GraphQL

Based on Lecture 8 (GraphQL) and the hands-on tutorial · Team submission

Goal. Take **one** small service from CampusEats and expose it through a GraphQL API — one schema, one query, one mutation, backed by MySQL or any other database— then demonstrate it using a simple HTML/CSS page.

TASK

Pick **any one** CampusEats entity — e.g. **Menu** (id, name, price, available), **Restaurants**, or **Orders**. Model it in GraphQL and corresponding resolvers over a MySQL/other database table. Follow the Books & Authors tutorial as your template.

#	Do this
1	**Schema** — Define one object type and a Query (list / by-id) plus one Mutation (add or update). Mark non-null fields with !.
2	**Database** — One MySQL table with 3–4 seed rows. Connect with mysql2, as in the tutorial.
3	**Resolvers** — Write the resolvers that run SQL for each query and the mutation. No business logic beyond the CRUD.
4	**Front-end** — A plain HTML/CSS page that fetch()es the GraphQL endpoint and lists the data; one button that fires the mutation.
5	**Run it** — Start the server, open the page, add/update one record live, and screenshot the before/after.

ONE QUERY + ONE MUTATION (minimum)

```
ask — list dishes, then add one

query  { dishes { id name price available } }

mutation {
  addDish(input: { name: "Paneer Roll", price: 90 }) {
    id name price
  }
}
```

DELIVERABLES

- `schema.graphql` — your type, Query and Mutation.
- `server.js + db.js` — Apollo Server with working resolvers over MySQL.
- `schema.sql` — table definition and seed rows.
- `public/index.html` — the page that reads and mutates the data.
- **A GitHub repo** with the above + **Google drive link with Demonstration video** + two screenshots (before / after the mutation) and a one-line README on how to run it.
- One submission per team via Google Classroom — repo link, with roll numbers and names on the README.

ANSWER IN THE README

1. Which CampusEats service did you pick?
2. Name one field a client can **skip** and one it **cannot** (why we use the "`!`" sign).
3. Map it to SQL/other database: which part of your query became the `WHERE`, and which resolver ran the `INSERT`?

Dr. Pramit Mazumdar · pramit.mazumdar@iiitvadodara.ac.in